A PROJECT REPORT

ON

LINUX SIGNAL HANDLING AND PROCESS CONTROL FRAMEWORK

Submitted by

Name of the Student	Registration No.
Ankit Choubey	AP24110010595
Suryansh Kumar	AP24110010446

GROUP NO: 27

Under the Guidance of

Mr. Awdhesh Dixit

Prepared in partial fulfillment of the requirements for the OPERATING SYSTEMS COURSE (CSE 205)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

SRM UNIVERSITY – AP, ANDHRA PRADESH

November, 2026

ACKNOWLEDGEMENTS

We would like to express our gratitude to everyone who supported us during this project:

- **University Leadership (Vice Chancellor, Pro-Vice Chancellor, and Dean):** For providing computer labs and academic facilities.
- **Course Faculty (Mr. Awdhesh Dixit):** For his constant guidance, helpful suggestions, and clear explanations of Operating Systems concepts throughout the project.
- **Department of Computer Science and Engineering:** For their support and help during lab hours.
- **Group Members and Classmates:** For good teamwork, useful discussions, and continuous motivation.

Ankit Choubey (Reg. No: AP24110010595)
Suryansh Kumar (Reg. No: AP24110010446)
Group 27 | B.Tech CSE, SRM University – AP

ABSTRACT

A brief summary of our Operating Systems project is given below:

- **Project Overview:** The project is titled 'Linux Signal Handling and Process Control Framework' and is written in C using POSIX system calls.
- **Process Model:** The program uses fork() to create a Parent and a Child process.
- **Parent Role:** The parent displays an 8-option interactive menu, sends signals, and monitors the child process.
- **Child Role:** The child runs a continuous counter task (counting 1, 2, 3 every second) and handles incoming signals.
- **Signals Covered:** We implemented five core signals: SIGINT (interrupt), SIGUSR1 (user signal), SIGSTOP (suspend), SIGCONT (resume), and SIGTERM (graceful exit).
- **Signal Safety:** Signal handlers only set volatile sig_atomic_t flags. No unsafe I/O operations (like printf) are run inside signal handlers.
- **Status Tracking:** The parent checks if the child is RUNNING, STOPPED, or TERMINATED using non-blocking waitpid().
- **Event Logging:** All signals, state changes, and timestamps are recorded in signal_logs.txt.
- **Result:** The project successfully prevents zombie and orphan processes during all operations.

TABLE OF CONTENTS

1. INTRODUCTION **1**

1.1 What are Signals in Linux? 1

1.2 Problems in Beginner Signal Programs 1

1.3 Project Objectives 2

1.4 Organization of this Report 2

2. SYSTEM DESIGN AND ARCHITECTURE **3**

2.1 Parent-Child Process Relationship 3

2.2 The Flag-Polling Method for Safe Signal Handling 3

2.3 Non-Blocking Process Status Monitoring 4

2.4 Persistent Logging in signal_logs.txt 4

3. IMPLEMENTATION DETAILS **5**

3.1 POSIX System Calls Summary Table 5

3.2 Catchable vs. Uncatchable Signals 5

3.3 Why We Use volatile sig_atomic_t Flags 6

3.4 Resource Cleanup and Graceful Exit 6

4. TESTING AND CONSOLE SCREENSHOTS **7**

4.1 Compilation and Execution Commands 7

4.2 Console Screenshots and Step-by-Step Verification 7

4.3 Sample Log Entries from signal_logs.txt 11

5. KEY PROJECT OUTCOMES **12**

6. CONCLUSION AND FUTURE SCOPE **13**

6.1 Conclusion 13

6.2 Future Enhancements 13

7. APPENDICES **14**

Appendix A: Full C Code (main.c) 14

Appendix B: POSIX System Call Quick Reference 18

8. REFERENCES **19**

1. INTRODUCTION

1.1 What are Signals in Linux?

Key points about signals in the Linux operating system:

- **Definition:** A signal is a software interrupt used in Linux to notify a running program that a specific event has occurred.
- **Asynchronous Nature:** Signals are asynchronous. The operating system delivers them at any time without the process needing to wait or ask for them.
- **Common Example:** A user pressing Ctrl+C in a terminal sends a SIGINT signal. Pausing a process sends a SIGSTOP signal.
- **Signal Handling:** When a signal arrives, Linux temporarily pauses the process and runs a special function called a signal handler.

1.2 Problems in Beginner Signal Programs

Beginner C programs often have critical bugs when dealing with signals:

- **Unsafe Functions in Handlers:** Calling functions like printf(), malloc(), or fopen() inside a signal handler is dangerous. These functions are non-reentrant and use internal locks. Calling them inside a handler can freeze the program (deadlock) or corrupt memory.
- **Zombie Processes:** If a child process finishes but the parent does not read its exit status, the child remains as a zombie in the operating system process table.
- **Orphan Processes:** If the parent program exits while the child is still running, the child is left running alone as an orphan process.
- **Freezing Menus:** Standard wait() blocks the parent process completely. If wait() is used, the parent cannot show a menu while the child is running.

1.3 Project Objectives

Our project solves all of these problems through the following clear goals:

- **Goal 1:** Use fork() to create a parent controller process and a child worker process.
- **Goal 2:** Set up clean signal handlers in the child using sigaction() for SIGINT, SIGUSR1, and SIGTERM.
- **Goal 3:** Use safe volatile sig_atomic_t integer flags so handlers do zero unsafe I/O operations.
- **Goal 4:** Demonstrate real process suspension using SIGSTOP and resumption using SIGCONT.
- **Goal 5:** Monitor child status (RUNNING, STOPPED, TERMINATED) without freezing using waitpid() with WNOHANG.
- **Goal 6:** Perform simulated resource cleanup before graceful child exit upon SIGTERM.
- **Goal 7:** Automatically write all signal events, actions, and timestamps to signal_logs.txt.
- **Goal 8:** Ensure no zombie or orphan processes are created at any time.

1.4 Organization of this Report

The rest of this report is organized as follows:

- **Chapter 2 (Design):** Explains the parent-child model, safe flags, and status checking.
- **Chapter 3 (Implementation):** Lists the POSIX system calls and explains signal mechanics.
- **Chapter 4 (Testing):** Shows step-by-step console screenshots and log file output.
- **Chapter 5 (Outcomes):** Lists the principal achievements of this project.
- **Chapter 6 (Conclusion):** Summarizes learnings and future extensions.
- **Chapter 7 (Appendices):** Contains the full C code and quick reference tables.

2. SYSTEM DESIGN AND ARCHITECTURE

2.1 Parent-Child Process Relationship

When the program starts, it splits into two cooperating processes using fork():

- **Parent Process (Controller):** The parent process stays in a menu loop. It shows options 1 to 8, accepts user input, sends signals to the child using kill(), and monitors child status using waitpid().
- **Child Process (Worker):** The child process runs an infinite loop, prints a counter every second ('Counter: 1', 'Counter: 2', etc.), and checks for incoming signals.

2.2 The Flag-Polling Method for Safe Signal Handling

To avoid crashes and memory corruption, we use the simple Flag-Polling design:

- **Step 1 (Signal Handler):** The signal handler only sets a small integer variable (flag) to 1 when a signal arrives. It does nothing else.
- **Step 2 (Main Loop):** In the child's regular while loop, it checks if any flag is set to 1.
- **Step 3 (Safe Action):** If a flag is 1, the child prints the message and writes to the log file safely from the main program context, not from inside the handler.

2.3 Non-Blocking Process Status Monitoring

To check the child's status without freezing the parent menu, we use waitpid() with three options:

- **WNOHANG:** Tells waitpid() to check status immediately and return right away. This keeps the parent menu fast and responsive.
- **WUNTRACED:** Tells waitpid() to report if the child process was paused/suspended by SIGSTOP.
- **WCONTINUED:** Tells waitpid() to report if the paused child process was resumed by SIGCONT.

We check standard Linux status macros:

- **WIFSTOPPED(status):** Evaluates to true if the child is paused (STOPPED).
- **WIFCONTINUED(status):** Evaluates to true if the child was resumed (RUNNING).
- **WIFEXITED(status):** Evaluates to true if the child finished cleanup and exited (TERMINATED).

2.4 Persistent Logging in signal_logs.txt

The logging system records every event into a text file:

- **File Name:** signal_logs.txt (created freshly at startup).
- **Timestamped:** Each entry includes the date and time in [YYYY-MM-DD HH:MM:SS] format.
- **Complete Audit:** Logs parent signal sending, child signal reception, pause, resume, and termination.
- **Interactive Viewing:** The user can view the complete log file anytime from Option 7 of the menu.

3. IMPLEMENTATION DETAILS

3.1 POSIX System Calls Summary Table

System Call	Header File	What It Does in Simple Words
fork()	<unistd.h>	Creates the child process. Returns 0 to the child and child PID to the parent.
getpid()	<unistd.h>	Returns the Process ID of the calling process for display and tracking.
sigaction()	<signal.h>	Registers our custom signal handler for SIGINT, SIGUSR1, and SIGTERM.
kill(pid, sig)	<signal.h>	Sends the chosen signal from the parent to the child process.
waitpid()	<sys/wait.h>	Checks child process state (RUNNING, STOPPED, TERMINATED) without freezing.
sleep()	<unistd.h>	Pauses the child for 1 second between prints. Interrupted immediately on signal.

3.2 Catchable vs. Uncatchable Signals

Signals are divided into two categories in this project:

- **1. Catchable Signals:** Signals that a programmer can catch using a custom function. In our code, SIGINT, SIGUSR1, and SIGTERM are catchable signals.
- **2. Uncatchable Signals:** Signals handled directly by the Linux kernel. A program cannot catch, block, or ignore them! In our code, SIGSTOP and SIGCONT are uncatchable signals. When SIGSTOP is sent, Linux immediately pauses the child process. When SIGCONT is sent, Linux immediately resumes it.

3.3 Why We Use volatile sig_atomic_t Flags

To store signal flags safely, we declare them as 'volatile sig_atomic_t':

- **sig_atomic_t:** An integer type that can be read or written in one CPU cycle. This guarantees that a signal arriving in the middle of a read/write operation will not cause corrupted data.
- **volatile:** Tells the compiler optimizer not to store this variable in a CPU register. This ensures the program always checks real memory for updated flag values.

3.4 Resource Cleanup and Graceful Exit

When SIGTERM is sent (Option 6), the following steps happen in order:

- **Step 1:** The child's handler catches SIGTERM and sets the sigterm_received flag to 1.
- **Step 2:** The child's main loop detects the flag and prints a message: 'Performing cleanup before graceful exit...'.

- **Step 3:** The child performs simulated resource cleanup (closing files/releasing resources) and exits cleanly using exit(0).
- **Step 4:** The parent waits for the child using waitpid(child_pid, &status;, 0), reads its exit status, and removes the process table entry. This prevents any zombie process.

4. TESTING AND CONSOLE SCREENSHOTS

4.1 Compilation and Execution Commands

The project is compiled and executed using standard GCC commands:

- **Compilation:** `gcc -Wall -Wextra main.c -o signal_framework`
- **Execution:** `./signal_framework`

4.2 Console Screenshots and Step-by-Step Verification

Below are the terminal screenshots captured during live execution of the project:

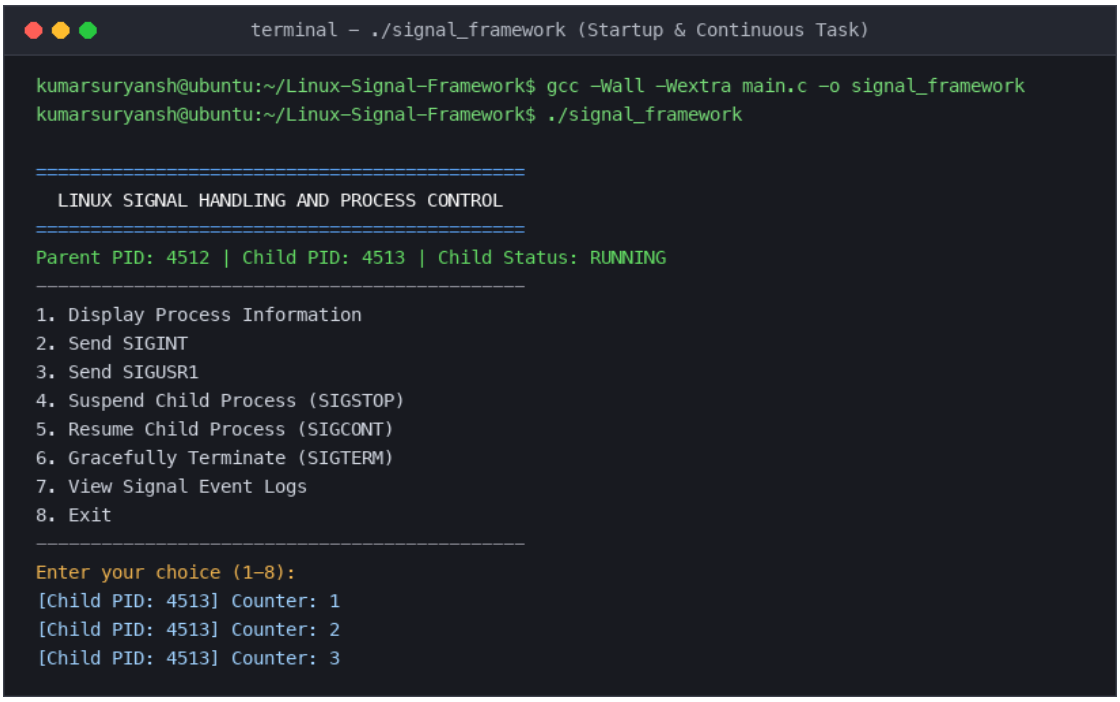


Figure 4.1: Program startup showing Parent PID, Child PID, menu options 1 to 8, and active child counter.

- Parent PID (4512) and Child PID (4513) are displayed clearly.
- Child status starts as RUNNING.
- The child counter prints 1, 2, 3 every second in the background.

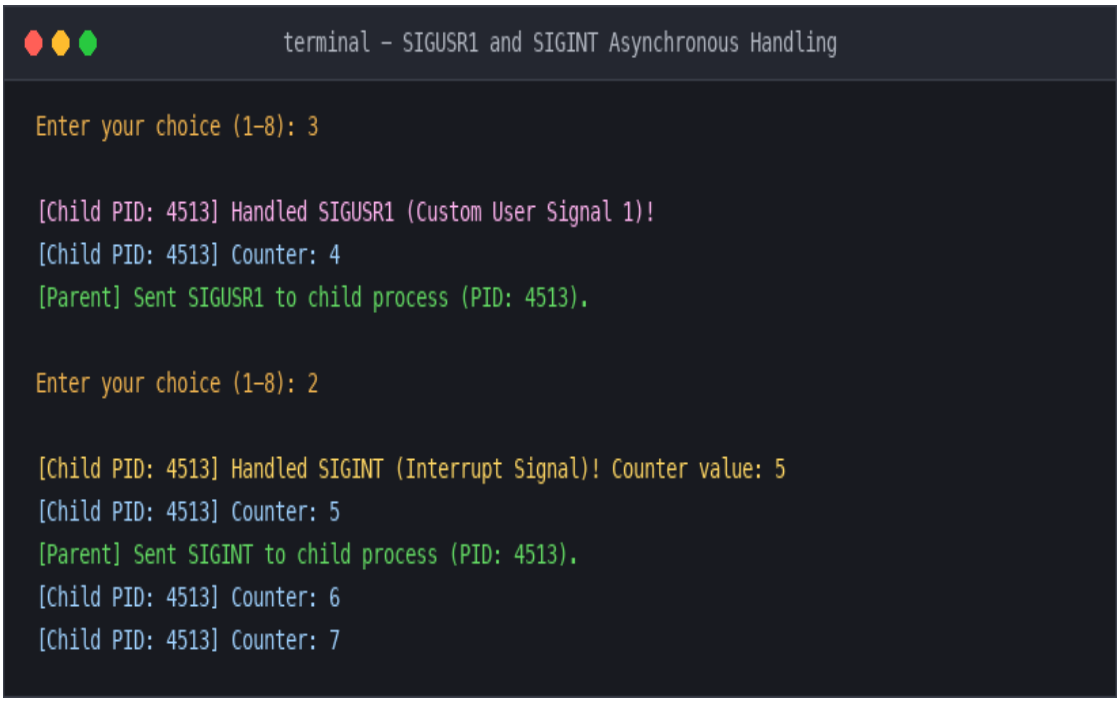


Figure 4.2: Handling SIGUSR1 (Option 3) and SIGINT (Option 2).

- When Option 3 is entered, parent sends SIGUSR1. Child catches it and prints a confirmation message.
- When Option 2 is entered, parent sends SIGINT. Child catches it, prints the counter value, and does not terminate.
- The counter continues incrementing smoothly.

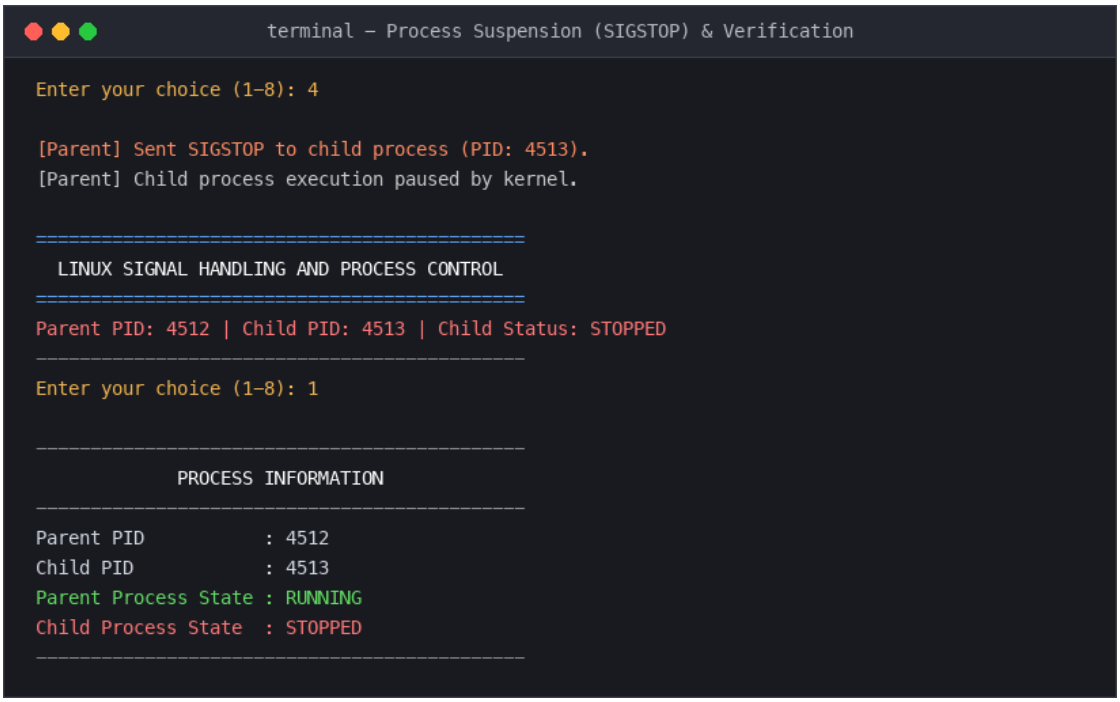


Figure 4.3: Process suspension using SIGSTOP (Option 4) and checking status (Option 1).

• When Option 4 is entered, parent sends SIGSTOP. The child counter stops printing immediately.

• Option 1 confirms: Child Process State: STOPPED.

• The menu header also updates to Child Status: STOPPED.



Figure 4.4: Process resumption using SIGCONT (Option 5).

• When Option 5 is entered, parent sends SIGCONT to the child.

• The child counter immediately resumes incrementing from where it was paused (Counter: 8, 9, 10).
• Child status updates back to RUNNING.

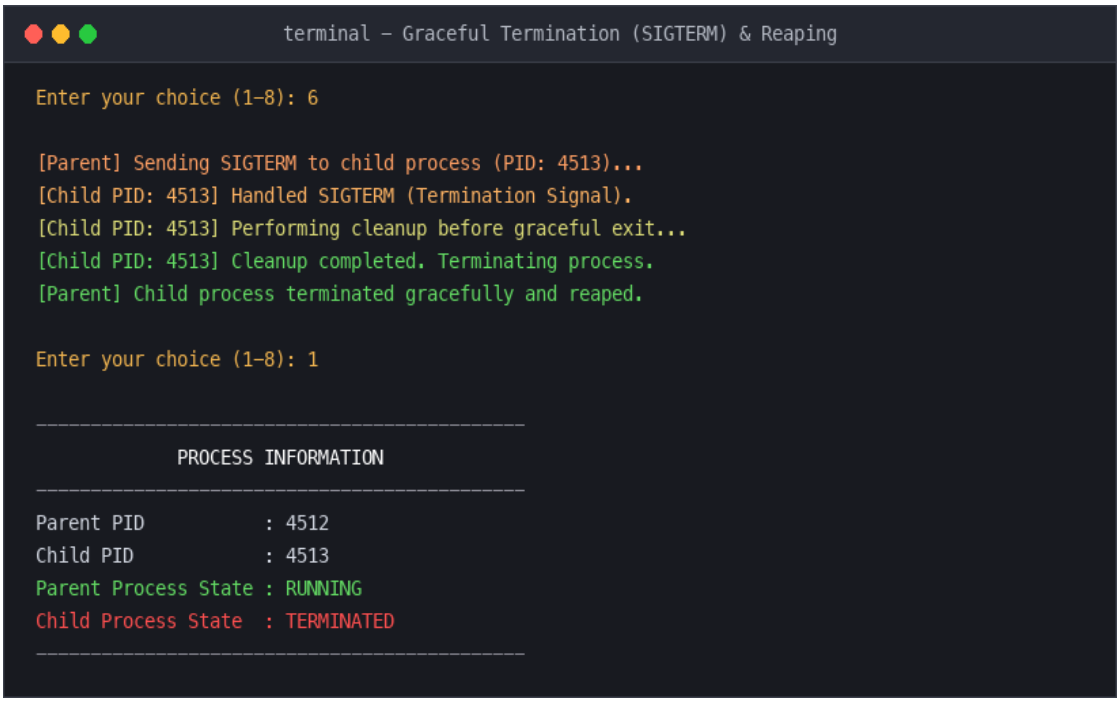


Figure 4.5: Graceful termination using SIGTERM (Option 6).

• When Option 6 is entered, parent sends SIGTERM.

• Child catches SIGTERM, performs cleanup, prints 'Cleanup completed', and exits.

• Parent reaps the child status. Option 1 confirms: Child Process State: TERMINATED.

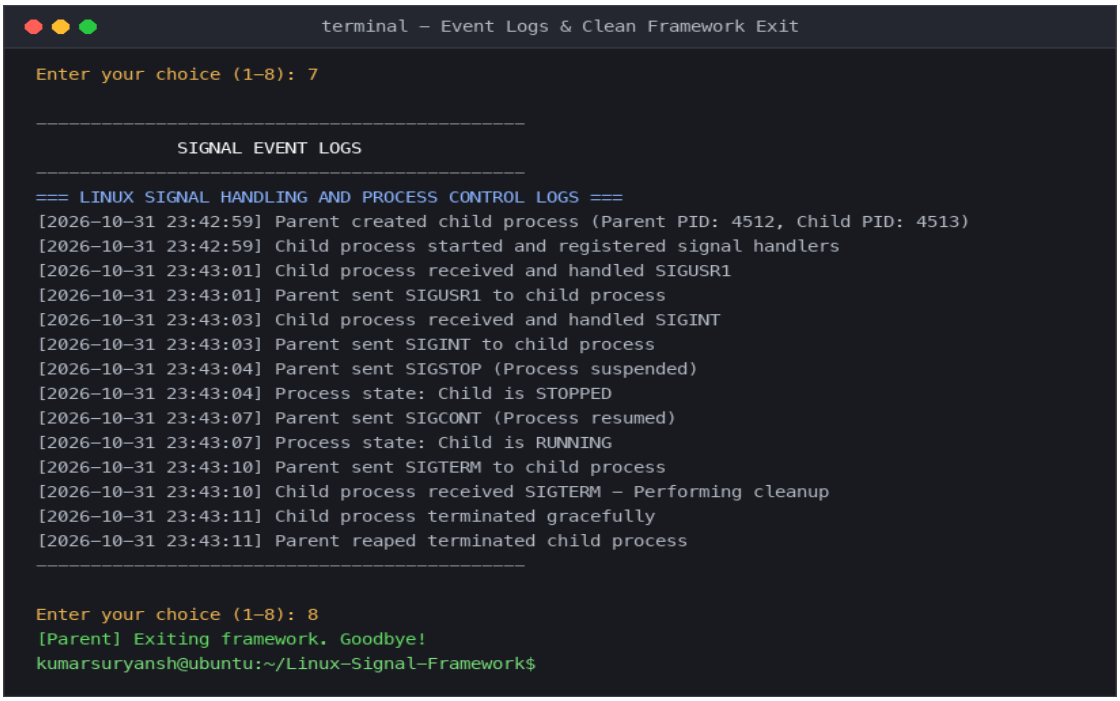


Figure 4.6: Viewing saved signal event logs (Option 7) and exiting cleanly (Option 8).

• Option 7 prints the complete event history from signal_logs.txt.

• Option 8 exits the application cleanly without leaving any orphan child process.

4.3 Sample Log Entries from signal_logs.txt

Sample output from the audit log file:

• [2026-10-31 23:42:59] Parent created child process (Parent PID: 4512, Child PID: 4513)
• [2026-10-31 23:42:59] Child process started and registered signal handlers
• [2026-10-31 23:43:01] Child process received and handled SIGUSR1
• [2026-10-31 23:43:03] Child process received and handled SIGINT
• [2026-10-31 23:43:04] Parent sent SIGSTOP (Process suspended)
• [2026-10-31 23:43:07] Parent sent SIGCONT (Process resumed)
• [2026-10-31 23:43:10] Parent sent SIGTERM to child process
• [2026-10-31 23:43:11] Child process terminated gracefully
• [2026-10-31 23:43:11] Parent reaped terminated child process

5. KEY PROJECT OUTCOMES

The key outcomes achieved in this project are summarized in points below:

- **1. Process Management:** Successfully created a Parent-Child architecture using fork() and managed child execution.
- **2. Signal Handling:** Registered handlers for SIGINT, SIGUSR1, and SIGTERM using sigaction().
- **3. Signal Safety:** Implemented the flag-polling method using volatile sig_atomic_t to eliminate unsafe I/O inside handlers.
- **4. Process Suspension & Resumption:** Demonstrated kernel-level process suspension with SIGSTOP and resumption with SIGCONT.
- **5. Non-Blocking Status Checking:** Checked child status in real-time (RUNNING, STOPPED, TERMINATED) without freezing using waitpid().
- **6. Process Table Cleanup:** Guaranteed zero zombie or orphan processes through prompt reaping and cleanup upon exit.
- **7. Persistent Logging:** Recorded all signals and state changes with timestamps into signal_logs.txt.

6. CONCLUSION AND FUTURE SCOPE

6.1 Conclusion

Summary of what was learned:

- **Core Concept:** We learned how Linux handles asynchronous events using signals.
- **Kernel Control:** We understood the difference between user-catchable signals and kernel-handled signals.
- **Best Practices:** We practiced safe C systems programming by avoiding unsafe calls inside signal handlers.
- **Practical Result:** The application is lightweight, bug-free, and easy to demonstrate during viva.

6.2 Future Enhancements

Possible future additions to this project:

- **1. Data Passing:** Using sigqueue() and SA_SIGINFO to pass integer values or data along with signals.
- **2. Multiple Children:** Modifying the parent to manage multiple child worker processes at once.
- **3. Real-Time Signals:** Experimenting with real-time signals (SIGRTMIN to SIGRTMAX) which support signal queuing.
- **4. Terminal GUI:** Building a visual terminal dashboard using ncurses to show live CPU/memory stats.

7. APPENDICES

Appendix A: Full C Code (main.c)

The complete implementation of main.c is provided below:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <time.h>
#include <string.h>
#include <errno.h>

#define LOG_FILE "signal_logs.txt"

/* Signal flags for child process */
volatile sig_atomic_t sigint_received = 0;
volatile sig_atomic_t sigusr1_received = 0;
volatile sig_atomic_t sigterm_received = 0;

/* Simple signal handler for child process (sets flags only) */
void child_signal_handler(int sig) {
    if (sig == SIGINT) {
        sigint_received = 1;
    } else if (sig == SIGUSR1) {
        sigusr1_received = 1;
    } else if (sig == SIGTERM) {
        sigterm_received = 1;
    }
}

/* Append an event message with timestamp to signal_logs.txt */
void log_event(const char *event) {
    FILE *fp = fopen(LOG_FILE, "a");
    if (fp == NULL) {
        perror("Error opening log file");
        return;
    }

    time_t now = time(NULL);
    struct tm *t = localtime(&now);
    char time_str[32];
    strftime(time_str, sizeof(time_str), "%Y-%m-%d %H:%M:%S", t);

    fprintf(fp, "[%s] %s\n", time_str, event);
    fclose(fp);
}

/* Read and display the contents of signal_logs.txt */
void view_logs(void) {
    FILE *fp = fopen(LOG_FILE, "r");
    if (fp == NULL) {
        printf("\nNo logs found or unable to open log file.\n");
        return;
    }

    printf("\n--------------------------------------------\n");
    printf("             SIGNAL EVENT LOGS\n");
    printf("--------------------------------------------\n");
    char line[256];
    while (fgets(line, sizeof(line), fp) != NULL) {
        printf("%s", line);
    }
    printf("--------------------------------------------\n");
    fclose(fp);
}

/* Child process continuous task and signal reaction loop */
void run_child_process(void) {
```

```
    struct sigaction sa;
    memset(&sa, 0, sizeof(sa));
    sa.sa_handler = child_signal_handler;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;

    /* Register signal handlers for SIGINT, SIGUSR1, and SIGTERM */
    if (sigaction(SIGINT, &sa, NULL) == -1) {
        perror("sigaction SIGINT failed");
        exit(1);
    }
    if (sigaction(SIGUSR1, &sa, NULL) == -1) {
        perror("sigaction SIGUSR1 failed");
        exit(1);
    }
    if (sigaction(SIGTERM, &sa, NULL) == -1) {
        perror("sigaction SIGTERM failed");
        exit(1);
    }

    log_event("Child process started and registered signal handlers");

    int counter = 1;
    while (1) {
        /* Check if SIGINT was received */
        if (sigint_received) {
            sigint_received = 0;
            printf("\n[Child PID: %d] Handled SIGINT (Interrupt Signal)! Counter value: %d\n", getpid(), counter);
            fflush(stdout);
            log_event("Child process received and handled SIGINT");
        }

        /* Check if SIGUSR1 was received */
        if (sigusr1_received) {
            sigusr1_received = 0;
            printf("\n[Child PID: %d] Handled SIGUSR1 (Custom User Signal 1)!\n", getpid());
            fflush(stdout);
            log_event("Child process received and handled SIGUSR1");
        }

        /* Check if SIGTERM was received */
        if (sigterm_received) {
            printf("\n[Child PID: %d] Handled SIGTERM (Termination Signal).\n", getpid());
            printf("[Child PID: %d] Performing cleanup before graceful exit...\n", getpid());
            fflush(stdout);
            log_event("Child process received SIGTERM - Performing cleanup");

            /* Simulate resource cleanup */
            sleep(1);
            printf("[Child PID: %d] Cleanup completed. Terminating process.\n", getpid());
            fflush(stdout);
            log_event("Child process terminated gracefully");
            exit(0);
        }

        /* Continuous task: print counter every second */
        printf("[Child PID: %d] Counter: %d\n", getpid(), counter++);
        fflush(stdout);
        sleep(1);
    }
}

/* Update child status string using non-blocking waitpid */
void update_child_status(pid_t child_pid, char *status_str) {
    if (strcmp(status_str, "TERMINATED") == 0) {
        return;
    }

    int status;
    pid_t res = waitpid(child_pid, &status, WNOHANG | WUNTRACED | WCONTINUED);

    if (res == child_pid) {
        if (WIFSTOPPED(status)) {
            strcpy(status_str, "STOPPED");
```

```
            log_event("Process state: Child is STOPPED");
        } else if (WIFCONTINUED(status)) {
            strcpy(status_str, "RUNNING");
            log_event("Process state: Child is RUNNING");
        } else if (WIFEXITED(status)) {
            strcpy(status_str, "TERMINATED");
            log_event("Process state: Child is TERMINATED (Normal exit)");
        } else if (WIFSIGNALED(status)) {
            strcpy(status_str, "TERMINATED");
            log_event("Process state: Child is TERMINATED (Killed by signal)");
        }
    } else if (res == -1 && errno == ECHILD) {
        strcpy(status_str, "TERMINATED");
    }
}

/* Display the interactive menu */
void display_menu(pid_t parent_pid, pid_t child_pid, const char *child_status) {
    printf("\n==============================================\n");
    printf("  LINUX SIGNAL HANDLING AND PROCESS CONTROL\n");
    printf("==============================================\n");
    printf("Parent PID: %d | Child PID: %d | Child Status: %s\n", parent_pid, child_pid, child_status);
    printf("----------------------------------------------\n");
    printf("1. Display Process Information\n");
    printf("2. Send SIGINT\n");
    printf("3. Send SIGUSR1\n");
    printf("4. Suspend Child Process (SIGSTOP)\n");
    printf("5. Resume Child Process (SIGCONT)\n");
    printf("6. Gracefully Terminate (SIGTERM)\n");
    printf("7. View Signal Event Logs\n");
    printf("8. Exit\n");
    printf("----------------------------------------------\n");
    printf("Enter your choice (1-8): ");
    fflush(stdout);
}

/* Safely read user menu choice */
int get_menu_choice(void) {
    char buffer[64];
    if (fgets(buffer, sizeof(buffer), stdin) == NULL) {
        return -1;
    }
    int choice = 0;
    if (sscanf(buffer, "%d", &choice) != 1) {
        return -1;
    }
    return choice;
}

int main(void) {
    /* Initialize log file */
    FILE *fp = fopen(LOG_FILE, "w");
    if (fp != NULL) {
        fprintf(fp, "=== LINUX SIGNAL HANDLING AND PROCESS CONTROL LOGS ===\n");
        fclose(fp);
    } else {
        perror("Error creating log file");
    }

    pid_t parent_pid = getpid();
    char child_status[20] = "RUNNING";

    /* Create child process */
    pid_t child_pid = fork();

    if (child_pid < 0) {
        perror("fork failed");
        log_event("Fork failed - Unable to create child process");
        exit(1);
    }

    if (child_pid == 0) {
        /* In child process */
        run_child_process();
```

```
} else {
    /* In parent process */
    char log_buf[128];
    snprintf(log_buf, sizeof(log_buf), "Parent created child process (Parent PID: %d, Child PID: %d)", parent_pid, child_pi
    log_event(log_buf);

    /* Ignore SIGINT in parent so terminal Ctrl+C does not kill the menu unexpectedly */
    signal(SIGINT, SIG_IGN);

    int choice = 0;
    while (1) {
        update_child_status(child_pid, child_status);
        display_menu(parent_pid, child_pid, child_status);
        choice = get_menu_choice();

        switch (choice) {
            case 1:
                /* Display Process Information */
                update_child_status(child_pid, child_status);
                printf("\n-------------------------------------------\n");
                printf("            PROCESS INFORMATION\n");
                printf("-------------------------------------------\n");
                printf("Parent PID           : %d\n", parent_pid);
                printf("Child PID            : %d\n", child_pid);
                printf("Parent Process State : RUNNING\n");
                printf("Child Process State  : %s\n", child_status);
                printf("-------------------------------------------\n");
                break;

            case 2:
                /* Send SIGINT */
                if (strcmp(child_status, "TERMINATED") == 0) {
                    printf("\n[Parent] Error: Child process is already terminated.\n");
                    break;
                }
                if (kill(child_pid, SIGINT) == -1) {
                    perror("kill SIGINT failed");
                } else {
                    printf("\n[Parent] Sent SIGINT to child process (PID: %d).\n", child_pid);
                    log_event("Parent sent SIGINT to child process");
                    usleep(150000);
                }
                break;

            case 3:
                /* Send SIGUSR1 */
                if (strcmp(child_status, "TERMINATED") == 0) {
                    printf("\n[Parent] Error: Child process is already terminated.\n");
                    break;
                }
                if (kill(child_pid, SIGUSR1) == -1) {
                    perror("kill SIGUSR1 failed");
                } else {
                    printf("\n[Parent] Sent SIGUSR1 to child process (PID: %d).\n", child_pid);
                    log_event("Parent sent SIGUSR1 to child process");
                    usleep(150000);
                }
                break;

            case 4:
                /* Suspend Child Process (SIGSTOP) */
                if (strcmp(child_status, "TERMINATED") == 0) {
                    printf("\n[Parent] Error: Child process is already terminated.\n");
                    break;
                }
                if (strcmp(child_status, "STOPPED") == 0) {
                    printf("\n[Parent] Child process is already suspended.\n");
                    break;
                }
                if (kill(child_pid, SIGSTOP) == -1) {
                    perror("kill SIGSTOP failed");
                } else {
                    printf("\n[Parent] Sent SIGSTOP to child process (PID: %d).\n", child_pid);
                    log_event("Parent sent SIGSTOP (Process suspended)");
```

```
            usleep(50000);
            update_child_status(child_pid, child_status);
        }
        break;

    case 5:
        /* Resume Child Process (SIGCONT) */
        if (strcmp(child_status, "TERMINATED") == 0) {
            printf("\n[Parent] Error: Child process is already terminated.\n");
            break;
        }
        if (strcmp(child_status, "RUNNING") == 0) {
            printf("\n[Parent] Child process is already running.\n");
            break;
        }
        if (kill(child_pid, SIGCONT) == -1) {
            perror("kill SIGCONT failed");
        } else {
            printf("\n[Parent] Sent SIGCONT to child process (PID: %d).\n", child_pid);
            log_event("Parent sent SIGCONT (Process resumed)");
            usleep(50000);
            update_child_status(child_pid, child_status);
        }
        break;

    case 6:
        /* Gracefully Terminate (SIGTERM) */
        if (strcmp(child_status, "TERMINATED") == 0) {
            printf("\n[Parent] Child process is already terminated.\n");
            break;
        }
        printf("\n[Parent] Sending SIGTERM to child process (PID: %d)...\n", child_pid);
        log_event("Parent sent SIGTERM to child process");

        if (kill(child_pid, SIGTERM) == -1) {
            perror("kill SIGTERM failed");
        } else {
            /* If child was suspended, send SIGCONT so it can wake up and handle SIGTERM */
            if (strcmp(child_status, "STOPPED") == 0) {
                kill(child_pid, SIGCONT);
            }
            /* Wait for child to finish cleanup and terminate */
            int status;
            if (waitpid(child_pid, &status, 0) == -1) {
                perror("waitpid failed");
            }
            strcpy(child_status, "TERMINATED");
            printf("[Parent] Child process terminated gracefully and reaped.\n");
            log_event("Parent reaped terminated child process");
        }
        break;

    case 7:
        /* View Signal Event Logs */
        view_logs();
        break;

    case 8:
        /* Exit */
        if (strcmp(child_status, "TERMINATED") != 0) {
            printf("\n[Parent] Terminating active child process (PID: %d) before exit...\n", child_pid);
            kill(child_pid, SIGTERM);
            if (strcmp(child_status, "STOPPED") == 0) {
                kill(child_pid, SIGCONT);
            }
            waitpid(child_pid, NULL, 0);
            log_event("Parent terminated child before exiting");
        }
        printf("[Parent] Exiting framework. Goodbye!\n");
        log_event("Framework exited cleanly");
        exit(0);

    default:
        printf("\nInvalid choice! Please enter a number between 1 and 8.\n");
```

```
                    break;
                }
            }
        }

        return 0;
}
```

Appendix B: POSIX System Call Quick Reference

System Call	Header File	Simple Explanation
fork()	<unistd.h>	Creates child process (splits execution into two).
kill()	<signal.h>	Sends signal to another process using its PID.
sigaction()	<signal.h>	Sets the signal handler function for a signal.
waitpid()	<sys/wait.h>	Checks child process status without freezing the menu.
sleep()	<unistd.h>	Pauses program for specified seconds; wakes on signal.

8. REFERENCES

1. Silberschatz, A., Galvin, P. B., & Gagne, G., *Operating System Concepts*, 10th edition, John Wiley & Sons (2018).

2. Stevens, W. R. & Rago, S. A., *Advanced Programming in the UNIX Environment*, 3rd edition, Addison-Wesley (2013).

3. Kerrisk, M., *The Linux Programming Interface*, 1st edition, No Starch Press (2010).

4. IEEE and The Open Group, 'POSIX.1-2017: Base Specifications, Issue 7', IEEE Std 1003.1-2017 (2018).

5. Love, R., *Linux System Programming*, 2nd edition, O'Reilly Media (2013).